



STrace

A DTrace on Windows Reimplementation

By Stephen Eckels

Senior Reverse Engineer - FLARE



Stephen Eckels

Senior Reverse Engineer (FLARE)

- Malware RE
- R&D - RE tooling

Experience

- Open-source development w/ a focus on hooking
- x86/x64 ASM, C++, Rust, Go, JS, C#, Java, more 😊
- System architect, embedded device software developer

Interests

- Slacklining
- Electric Mountainboarding
- Gaming



 @stevemk14ebr

 github.com/stevemk14ebr/

Presentation Outline

What is DTrace?

- Why is it interesting
- Usage of Microsoft's Port
- Shortcomings

DTrace on Windows Internals

- Modern dtrace architecture
- How kernel registration works
- Documenting kernel interfaces
- Inserting ourselves into the boot process
- Accessing paged out memory safely

STrace – Steve's Tracer

- Differences to Microsoft's port
- Writing a driver in Rust
- Adding WASM to the kernel
- Ditching rust drivers for C++
- Loading DLLs in drivers
- DIY kernel -> user callstack trace
- DIY callstack symbolication

DTrace Introduction

What it is

- Sun Microsystems -> Oracle -> OpenDTrace (2016)
- Tracing system allowing users to register probes
- Probes 'fire' on specific, user defined conditions
- Multiple types of probes are supported, each offering a different source of information
 - STP – syscalls hooks
 - FBT – kernel function boundary tracing
 - PID – user function boundary tracing
 - ETW – event tracing for windows (ETW) events
- When fired, probes execute user scripts in a VM inside the kernel

Examples

Print created files for a target process

```
syscall::NtCreateFile:entry
/ pid == $target && arg0 != 0 && arg2 !=0 /
{
    attr = (OBJECT_ATTRIBUTES*)copyin(arg2, sizeof(OBJECT_ATTRIBUTES));
    objectName = (UNICODE_STRING*)copyin(attr->ObjectName, sizeof(UNICODE_STRING));
    name = *((ustr*)copyin((uintptr_t)objectName->Buffer, objectName->Length));
    printf("OBJECT_NAME: %ws", name);
    ustack();
}
```

Live system dump on specific error code

```
syscall:::return {
    if (arg0 != 0xc0000001UL) {
        lkd(0);
        printf("\n Triggering Live dump \n \n");
        exit(0);
    }
}
```

Advanced DTrace Examples

The pain begins...

Deferring Reads

```
syscall::NtCreateFile:entry {  
    // data behind pointer may be paged out. Don't read until paged in.  
    self->pFileHandle = args[0];  
}  
  
syscall::NtCreateFile:return {  
    // system routine has paged in the memory.  
    if(arg0 == STATUS_SUCCESS) {  
        handle = *(HANDLE*)copyin(self->pFileHandle, sizeof(HANDLE));  
        printf("%08X", handle);  
    }  
}
```

Pretty print enum types.

```
syscall::NtCreateFile:entry  
/ pid == $target && arg0 != 0 && arg2 != 0 /  
{  
    this->access = "|";  
    this->access = strjoin(this->access, (arg1 & FILE_READ_DATA) ? "ReadData|": "" );  
    this->access = strjoin(this->access, (arg1 & FILE_WRITE_DATA) ? "WriteData|": "" );  
    this->access = strjoin(this->access, (arg1 & FILE_APPEND_DATA) ? "AppendData|": "" );  
    this->access = strjoin(this->access, (arg1 & FILE_READ_EA) ? "ReadEa|": "" );  
    this->access = strjoin(this->access, (arg1 & FILE_WRITE_EA) ? "WriteEa|": "" );  
    this->access = strjoin(this->access, (arg1 & FILE_EXECUTE) ? "Execute|": "" );  
    this->access = strjoin(this->access, (arg1 & FILE_DELETE_CHILD) ? "DeleteChild|": "" );  
    this->access = strjoin(this->access, (arg1 & FILE_READ_ATTRIBUTES) ? "ReadAttributes|": "" );  
    this->access = strjoin(this->access, (arg1 & FILE_WRITE_ATTRIBUTES) ? "WriteAttributes|": "" );  
    printf("CreateFile::ACCESS_MASK: %s", this->access);  
}
```

Advanced DTrace Examples

The pain begins...

Copying UNICODE STRINGS, C preprocessor for 'functions'

```
// DTCompile.bat
// @echo off
// start /wait cmd /c "cl.exe /P %1 && dtrace -s %~dpn1.i -c %2" & del %~dpn1.i
// Use: DTCompile.bat createfile.d %windir%/system32/notepad.exe

syscall::NtCreateFile:entry
/ pid == $target && arg0 != 0 && arg2 != 0 /
{
    copy_ustr(arg2, fname, fnamelen);
    printf("%*ws", fmt_ustr(fname, fnamelen));
}

#define copy_ustr(argname, varname, lenname) \
    attr = (OBJECT_ATTRIBUTES*)copyin(argname, sizeof(OBJECT_ATTRIBUTES)); \
    if(attr) { \
        objectName = (UNICODE_STRING*)copyin((uintptr_t)attr->ObjectName, sizeof(UNICODE_STRING)); \
        if (objectName) { \
            this->##varname = (struct ustr*)copyin((uintptr_t)objectName->Buffer, sizeof(struct ustr)); \
            this->##lenname = objectName->Length; \
        } \
    }

#define fmt_ustr(varname, lenname) (this->lenname / 2), this->varname->buffer
```

DTrace shortcomings

- D language Issues
 - No user defined functions!
 - No loops!
 - Control flow structures like ifs occasionally compile to invalid instruction sequences in some cases
- System design issues
 - Extremely limited side-effects. DTrace APIs like **system** only accept static strings
 - Issues accessing memory before syscalls execute
 - Features like the **dtrace** pre-processor are simply not implemented in Microsofts' port
 - Wide string support is shaky at best. No string utilities are provided at all.
 - Developer experience tools lacking entirely, no IDE, syntax highlighting, etc.

DTrace on Windows Internals

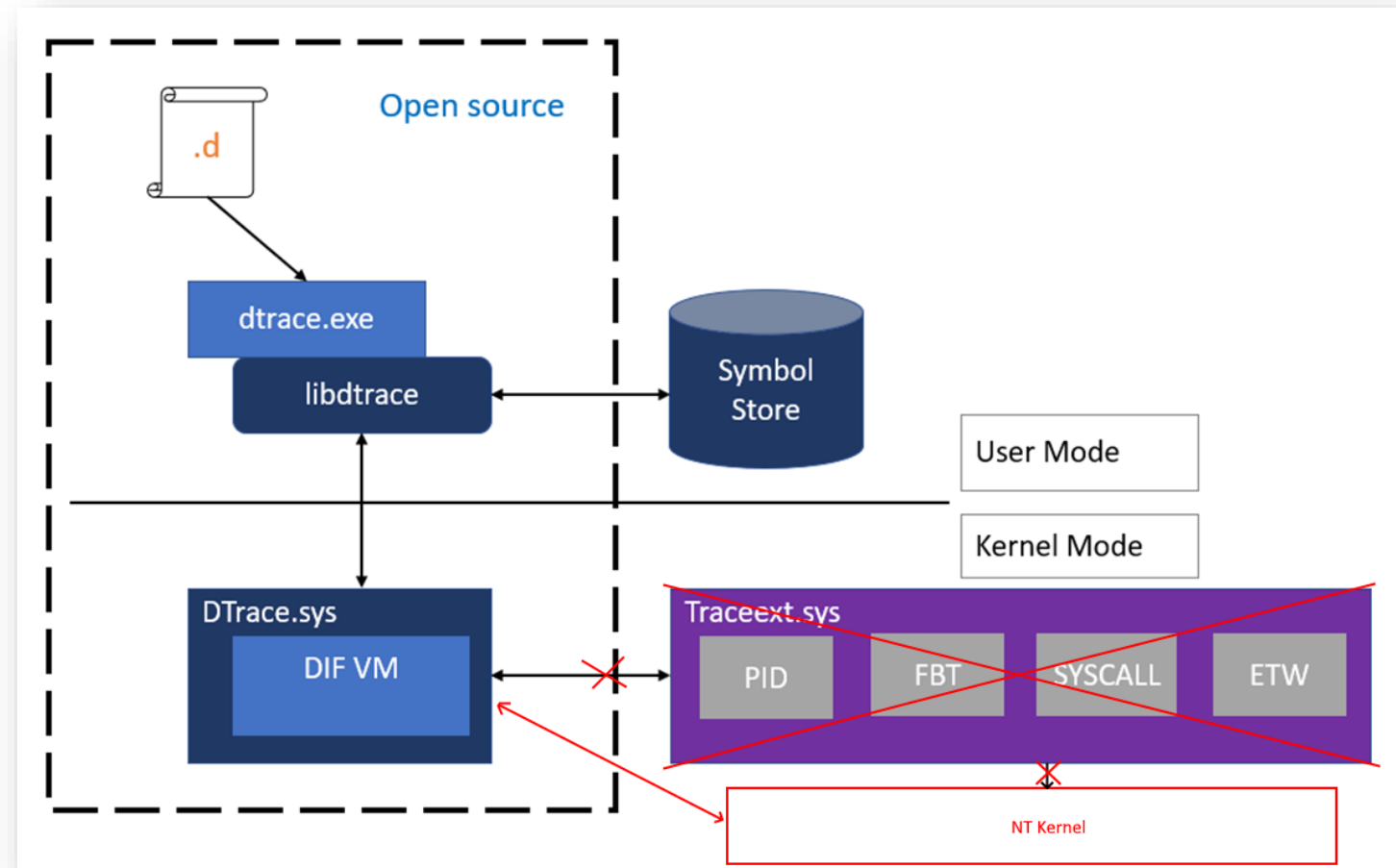
The start of a re-implementation

- High level architecture
- Boot process
- Neat dtrace kernel tricks
- Documenting kernel interfaces

We can do better!
<403 – meme forbidden>

Architecture

- The interface changed with 20H1! **traceext.sys** is no more
- **dtrace.sys** registers a set of function pointers with the kernel for each probe type
- **dtrace.sys** executes D language scripts in a VM and collects logs and tracing data
- **dtrace.exe** communicates with **dtrace.sys** to fetch queued tracing data and provides cached symbol information to the driver.



DTrace Boot

- The boot process starts at the bootloader **winload.exe**
 - Check boot flag **dtrace** is on (bcdedit)
 - Add **ext-ms-win-ntos-trace-l1-1-0.dll** to schema set map ①

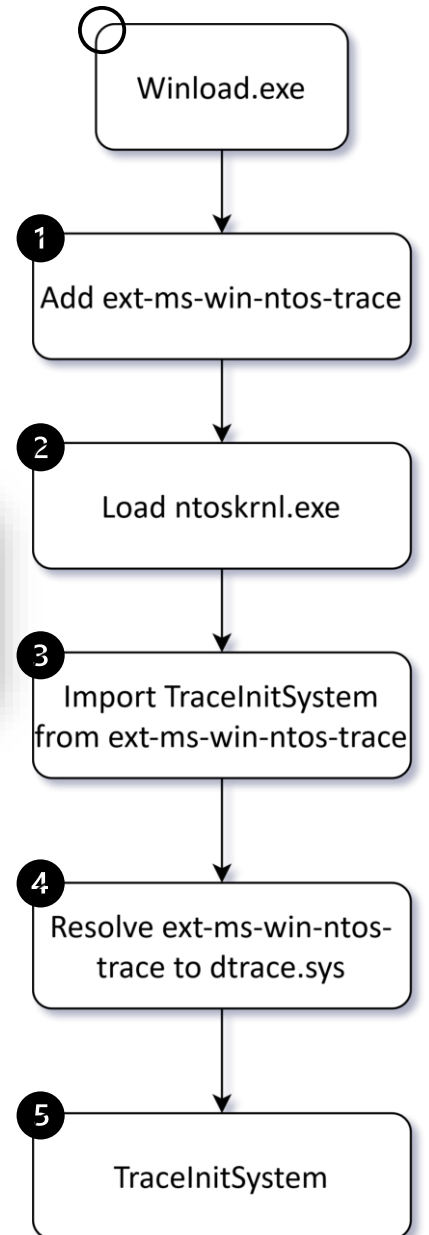
```
OsLpDynamicSchemaExtensions dq offset aExtMsWinNtosTr
                                ; DATA XREF: OsLpApplyDynamicSchemaExtensions+
                                ; OsLpApplyDynamicSchemaExtensions+20fo
                                ; "ext-ms-win-ntos-trace-l1-1-0"
dq offset aDtraceSys           ; "dtrace.sys"
dq offset OsLpDtraceExtensionEnabled
```

```
1 struct DynamicSchemaExtension
2 {
3     char *extension_dll_name;
4     char *extension_driver_name;
5     void *pFnVerifyBCDFlag;
6 };
```

- **winload.exe** begins to load **ntoskrnl.exe** ②
- **ntoskrnl.exe** imports from this apiset, consulting the schema ③ ④ set map

Address	Ordinal	Name	Library
00000000140131548		TraceInitSystem	ext-ms-win-ntos-trace-l1-1-0

- **ntoskrnl.exe** calls **TraceInitSystem** from **dtrace.sys** ⑤
 - Invoked during phase 1 kernel initialization, before patchguard
 - **KelInitSystem** -> **KilnitDynamicTraceSupport** -> **TraceInitSystem**

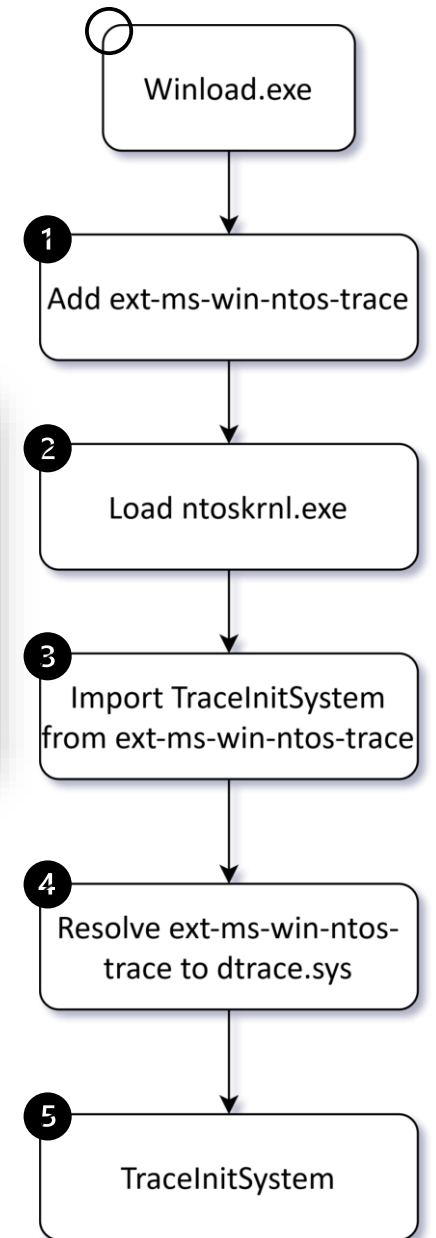


ApiSet Dlls

- 'Pseudo' PEs that contain a binary map pointing to implementing PEs

a6811726-50f2-43b1-a328-27285									
Name	Virtual Size	Virtual Address	Raw Size	Raw Address	Reloc Address	Linenumbers	Relocations N...	Linenumbers ...	Characteristics
Byte[8]	Dword	Dword	Dword	Dword	Dword	Dword	Word	Word	Dword
.rdata	000001A0	00001000	00000200	00000400	00000000	00000000	0000	0000	40000040
.apiset	0000009C	00002000	00000200	00000600	00000000	00000000	0000	0000	40000040
.rsrc	000004A8	00003000	00000600	00000800	00000000	00000000	0000	0000	40000040

- HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\Session Manager\ApiSetSchemaExtensions\<anything>
 - FileName=<path_to_api_set.dll>
 - Name=<anything>



apit_set.bin x

Edit As: Hex
Run Script
Run Template: Template1.bt

0123456789ABCDEF
0000h: (06 00 00 00) 9C 00 00 00 00 00 00 00 01 00 00 00 (. . . .) e
0010h: 1C 00 00 00 94 00 00 00 1F 00 00 00 00 00 00 00 00 "
0020h: 34 00 00 00 14 00 00 00 14 00 00 00 48 00 00 00 4 H
0030h: 01 00 00 00 64 00 74 00 72 00 61 00 63 00 65 00 d . t . r . a . c . e .
0040h: 2E 00 73 00 79 00 73 00 00 00 00 00 00 00 00 00 . . s . y . s
0050h: 00 00 00 00 5C 00 00 00 38 00 00 00 65 00 78 00 \ . . . 0 . . . e . x .
0060h: 74 00 2D 00 6D 00 73 00 2D 00 77 00 69 00 6E 00 t . - . m . s . - . w . i . n .
0070h: 2D 00 6E 00 74 00 67 00 73 00 2D 00 74 00 72 00 - . n . t . o . s . - . t . r .
0080h: 61 00 63 00 65 00 2D 00 6C 00 31 00 2D 00 31 00 a . c . e . - . l . l . - . l .
0090h: 2D 00 30 00 C0 9E CA 97 00 00 00 00 00 00 00 00 - . 0 . a z e -
00A0h: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .
00B0h: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .
00C0h: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .
00D0h: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .
00E0h: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .
00F0h: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .
0100h: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .
0110h: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .
0120h: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .

Template Results - Template1.bt

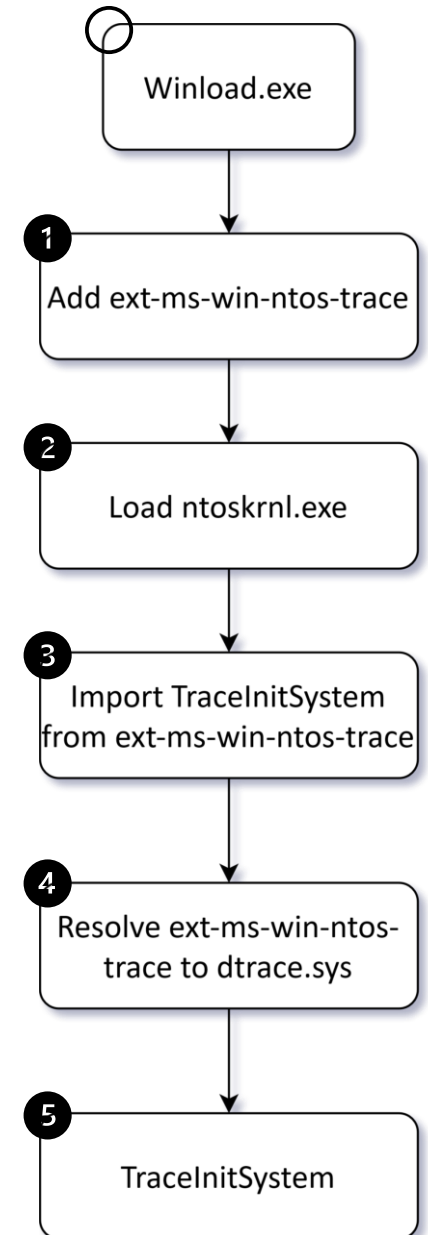
Name	Value	Start	Size	Color	Comment
struct API_SET_NAMESPACE ApiSetMap		0h	1Ch	Fg: Bg:	
ULONG Version	6	0h	4h	Fg: Bg:	
ULONG Size	9Ch	4h	4h	Fg: Bg:	
ULONG Flags	0	8h	4h	Fg: Bg:	
ULONG Count	1h	Ch	4h	Fg: Bg:	
ULONG EntryOffset	1Ch	10h	4h	Fg: Bg:	
ULONG HashOffset	94h	14h	4h	Fg: Bg:	
ULONG HashFactor	1Fh	18h	4h	Fg: Bg:	
struct API_SET_NAMESPACE_ENTRIES Entries		1Ch	18h	Fg: Bg:	
struct API_SET_NAMESPACE_ENTRY Entry		1Ch	18h	Fg: Bg:	
ULONG Flags	0	1Ch	4h	Fg: Bg:	
ULONG NameOffset	52	20h	4h	Fg: Bg:	
ULONG NameLength	20	24h	4h	Fg: Bg:	
ULONG HashedLength	20	28h	4h	Fg: Bg:	
ULONG ValueOffset	72	2Ch	4h	Fg: Bg:	
ULONG ValueCount	1	30h	4h	Fg: Bg:	
struct API_SET_ENTRY_PARSED ParsedEntry	dtrace.sys -> ext-ms-win-ntos-trace-l1-1-0	34h	68h	Fg: Bg:	
CHAR Name[22]	d	34h	16h	Fg: Bg:	
struct API_SET_VALUE_ENTRY ValueEntry		48h	14h	Fg: Bg:	
ULONG Flags	0	48h	4h	Fg: Bg:	
ULONG NameOffset	0	4Ch	4h	Fg: Bg:	
ULONG NameLength	0	50h	4h	Fg: Bg:	
ULONG ValueOffset	92	54h	4h	Fg: Bg:	
ULONG ValueLength	56	58h	4h	Fg: Bg:	
struct API_SET_ENTRY_VALUE ValueName		5Ch	3Ah	Fg: Bg:	
CHAR Value[58]	e	5Ch	3Ah	Fg: Bg:	
struct API_SET_NAMESPACE_HASH_ENTRIES HashEntries		94h	8h	Fg: Bg:	
struct API_SET_HASH_ENTRY HashEntry		94h	8h	Fg: Bg:	
ULONG Hash	2546638528	94h	4h	Fg: Bg:	
ULONG Index	0	98h	4h	Fg: Bg:	

ApiSetExtensions are flipped
Host -> ApiSet

Signing Requirements

- **dtrace.sys** is loaded as an import of **ntoskrnl.exe**
 - Despite being a dynamic extension, it's considered 'part of' the kernel
 - Must be signed with an MS root code signing cert
- schema set DLL must also be MS signed
- So...how do we load our reimplementation of **dtrace.sys**?
 - We can re-use the MS signed schema set DLL to enable the dynamic schema set extension
 - Create new boot entry to disable DSE so that **ntoskrnl.exe** can load our unsigned driver as an import
- Signing it yourself, test signing, custom kernel signers, etc...will **NOT** work.
 - Microsoft: Please reconsider this design and allow **dtrace.sys** to be signed by non-MS parties.

```
1  ImageValidateImageHash
2  ValidationFlags=IMGP_LATEST_MS_ROOT_REQUIRED
3  | IMGP_WINDOWS_ROOT_REQUIRED
4  | IMGP_MS_SIGNATURE_REQUIRED
5  Scenario=ImgSigningScenarioWindows
```

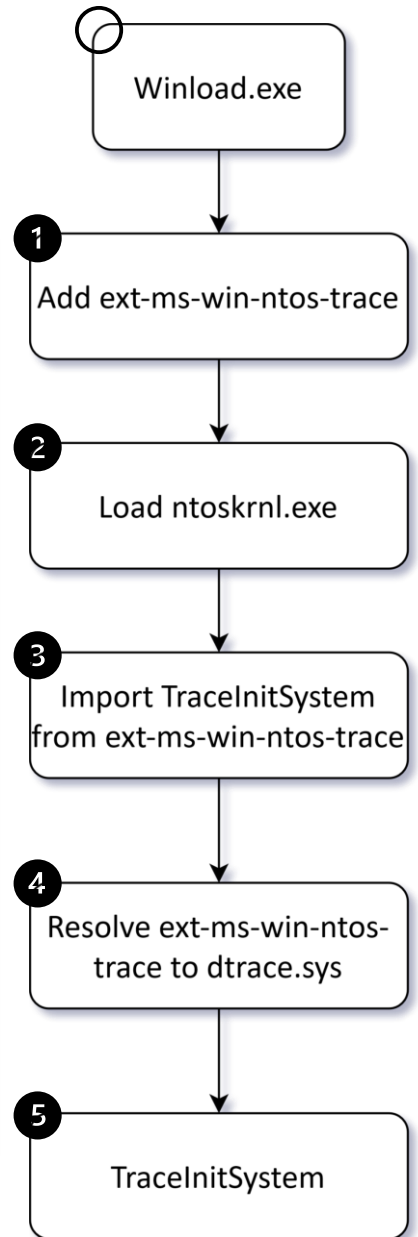


DTrace Kernel Registration

TraceInitSystem

- **dtrace.sys** fills **KiDynamicTraceCallouts** table to handle each probes' events
- **dtrace.sys** passes a pointer to **TraceAccessMemory** to **ntoskrnl.exe**
 - **ntoskrnl.exe** records the start and end bounds of this routine via the x64 unwind table entry
- **ntoskrnl.exe** passes a pointer to **KiDynamicTraceContext** to **dtrace.sys**
 - Provides offsets into kernel structures for the current kernel version
 - Provides APIs to set and remove trace probes

```
1 struct KiDynamicTraceContext
2 {
3     ULONG32 supported_flags;
4     ULONG32 kthread_tracingprivatedata_offset;
5     ULONG32 kthread_tracingprivatedata_arraysize;
6     ULONG32 kthread_trapframe_offset;
7     ULONG64 kthread_teb_offset;
8     NTSTATUS (*KeSetSystemServiceCallback)
9         (const char *syscallName, BOOLEAN isEntry,
10          ULONG64 pFnCallback, ULONG64 probeId);
11
12     void *KeSetTracepoint;
13     void *EtwRegisterEventCallback;
14     void *PsGetBaseTrapFrame;
15     void *KiGetTrapFrameRegister;
16     void *MmEnumerateSystemImages;
17 };
18
19 BOOLEAN TraceAccessMemory(
20     PVOID SafeAddress, ULONG_PTR UnsafeAddress,
21     SIZE_T NumberOfBytes, SIZE_T ChunkSize,
22     BOOLEAN DoRead);
23
24 struct KiDynamicTraceCallouts
25 {
26     void* pFnCallbacks[9];
27 };
28
```



Syscall Probes!

- **KiSystemCall64**
 - Fires for all usermode syscalls
 - And **Zw*** calls, as they go through **KiServiceInternal**
 - **Nt*** calls are not traced.
 - Shadow SSDT syscalls *not* supported
- **KiTrackSystemCallEntry** -> **StpCallbackEntry**
- **KiTrackSystemCallExit** -> **StpCallbackReturn**
- Probe entry / exit callbacks must be one of the two **Stp** functions set in **KiDynamicTraceCallouts** during **TraceInitSystem**
 - **KiDynamicTraceCallouts** is PG protected.

```
1 KiSystemCall64(syscallIdx) {  
2     pService = LookupServiceRoutineFromSSDT(syscallIdx);  
3     // ...  
4     if(KiDynamicTraceMask & AT_LEAST_ONE_SET) {  
5         machineContext = SnapshotArgAndStack();  
6         TreeEntry = KiTrackSystemCallEntry(pService, machineContext);  
7         return_value = pService(machineContext);  
8         KiTrackSystemCallExit(TreeEntry, return_value);  
9     }  
10    // ...  
11    return return_value;  
12 }
```

```
NTSTATUS SetCallbackApi(const char* syscallName, BOOLEAN isEntry, ULONG64 probeId) {  
    if (!TraceSystemApi || !TraceSystemApi->KeSetSystemServiceCallback) {  
        return STATUS_UNSUCCESSFUL;  
    }  
  
    auto callback = (ULONG64)(isEntry ? (&StpCallbackEntry) : (&StpCallbackReturn));  
    return TraceSystemApi->KeSetSystemServiceCallback(syscallName, isEntry, callback, probeId);  
}  
  
NTSTATUS UnSetCallbackApi(const char* syscallName, BOOLEAN isEntry) {  
    if (!TraceSystemApi || !TraceSystemApi->KeSetSystemServiceCallback) {  
        return STATUS_UNSUCCESSFUL;  
    }  
  
    return TraceSystemApi->KeSetSystemServiceCallback(syscallName, isEntry, 0, 0);  
}
```

DTrace Tricks

- **TraceAccessMemory** – safely read arbitrary inaccessible or paged out memory at a high IRQL w/o blue screening
 - **copyin** in DTrace scripts use this

```
result = TraceInitSystem(&ppTraceApi, &KiDynamicTraceCallouts, &TraceAccessMemoryStart);
if ( (result & 0x80000000) != 0 )
    return result;
pTraceFunc = RtlLookupFunctionEntry(TraceAccessMemoryStart, &imageBase, 0i64);
if ( pTraceFunc )
    // read memory routine bounds to detect memory faults
    // originating within this [start, end) range.
{
    TraceAccessMemoryStart = imageBase + pTraceFunc->BeginAddress;
    TraceAccessMemoryEnd = imageBase + pTraceFunc->EndAddress;
}
```

```
1 bool __fastcall KiIsTraceMemoryAccess(unsigned __int64 a1)
2 {
3     return KiDynamicTraceEnabled && a1 >= TraceAccessMemoryStart && a1 < TraceAccessMemoryEnd && KeGetCurrentIrql() == 15;
4 }
```

Why?

- **MmAccessFault** -> **KeInvalidAccessAllowed**

```
... 322         if (KeInvalidAccessAllowed (TrapInformation) == TRUE) {
323             return STATUS_ACCESS_VIOLATION;
324         }
325
326         KeBugCheckEx (PAGE_FAULT_IN_NONPAGED_AREA,
327                     (ULONG_PTR)VirtualAddress,
328                     FaultStatus,
329                     (ULONG_PTR)TrapInformation,
330                     7);
```


DTrace Tricks

- **self** keyword– kernel thread local storage

```
1 struct _KTHREAD
2 {
3     // ...
4     ULONGLONG TracingPrivate[1]; // 0x3c0
5     // ...
6 };
7
8 struct KiDynamicTraceContext
9 {
10    ULONG32 supported_flags;
11    ULONG32 kthread_tracingprivatedata_offset;
12    ULONG32 kthread_tracingprivatedata_arraysize;
13    // ...
14 };
```

```
void setTlsValue(uint64_t value) {
    PKTHREAD pThread = KeGetCurrentThread();

    if (kthread_tracingprivatedata_arraysize <= 0) {
        __debugbreak();
        return;
    }

    uint64_t* pTlsArray = (uint64_t*)((char*)pThread + kthread_tracingprivatedata_offset);
    pTlsArray[0] = value;
}

uint64_t getTlsValue() {
    PKTHREAD pThread = KeGetCurrentThread();

    if (kthread_tracingprivatedata_arraysize <= 0) {
        __debugbreak();
        return 0;
    }

    uint64_t* pTlsArray = (uint64_t*)((char*)pThread + kthread_tracingprivatedata_offset);
    return pTlsArray[0];
}
```

- Used by **dtrace** to cache D language intermediate data, such as thread local data

DTrace Tricks

- **dtrace_action_lkd** – live kernel dumps
 - **lkd** in DTrace scripts use this
- void **DbgWerCaptureLiveKernelDump**(wchar_t* ComponentName, ULONG BugCheckCode, ULONG P1, ULONG P2, ULONG P3, ULONG P4, ULONG flags);
 - Generates standard .dmp files loadable in windbg
 - NtSystemDebugControl
- Flags and registry settings to control this API now officially documented!
 - <https://docs.microsoft.com/en-us/windows-hardware/drivers/devtest/dtrace-live-dump>
- Generated dumps typically in C:\Windows\LiveKernelReports

DTrace Tricks

- **TraceWalkUserStack** – Generate a symbolic stack trace down to usermode.
 - **ustack()** in **dtrace**
- Operates differently from how ‘other’ tools generate stack traces
 - Usermode API: **StackWalk64** + DIA symbol APIs
 - Small trick to start at kernel, grab `Ethread.Kthread.KernelStack.RSP`
 - Trace initiated from usermode, drivers can’t call this
- **TraceWalkUserStack** does what **StackWalk64** does, manually
 - Trace initiated from kernelmode
 - **RtlCaptureStackBackTrace** doesn’t include usermode calls
- How?
 - Given some starting address, locate the module’s
 - `DataDirectory[IMAGE_DIRECTORY_ENTRY_EXCEPTION]`

```
typedef struct _RUNTIME_FUNCTION {
    ULONG BeginAddress;
    ULONG EndAddress;
    ULONG UnwindData;
} RUNTIME_FUNCTION, *PRUNTIME_FUNCTION;
```

“How to Unwind”

- unwind data defines a series of opcodes to ‘undo’ the effects of a function on the CPU state. These are ‘virtually’ executed, then the return address from the unwound stack is read, and the process repeats

Ex: `UWOP_PUSH_NONVOL` – a register pushed to stack, `RSP += 8` to undo

ProcMon

Stack - thread 11768

Name		Stack Walk Kernel UserMode
0	ntoskrnl.exe!KeWaitForSingleObject+0x38f0	
1	ntoskrnl.exe!KeWaitForSingleObject+0x1787	
2	ntoskrnl.exe!KeWaitForSingleObject+0x98f	
3	ntoskrnl.exe!KeWaitForSingleObject+0x233	
4	ntoskrnl.exe!ExWaitForRunDownProtectionRelease+0x7dd	
5	ntoskrnl.exe!KeWaitForSingleObject+0x3a29	
6	ntoskrnl.exe!KeSynchronizeExecution+0x3140	
7	vid.dll!VidVsmPrecommitMgmtVtlPageRange+0x1a6	
8	vid.dll!VidVsmPrecommitMgmtVtlPageRange+0x211	
9	vid.dll!VidVsmPrecommitMgmtVtlPageRange+0x237	
10	vid.dll!VidDispatchVirtualProcessor+0x8a	
11	WinHvPlatform.dll+0xf803	
12	WinHvPlatform.dll!WHvRunVirtualProcessor+0x2c	
13	vmware-vmx.exe+0x211e7	
14	vmware-vmx.exe+0x4efed	
15	vmware-vmx.exe+0x183cc	
16	vmware-vmx.exe+0x58e0d9	
17	vmware-vmx.exe+0x71d124	
18	kernel32.dll!BaseThreadInitThunk+0x14	
19	ntdll.dll!RtlUserThreadStart+0x21	

Latest Call

Oldest Call

STrace

Writing a re-implementation!

- Differences to Microsoft's port
- Writing a driver in Rust
- Adding WASM to the kernel
- Ditching rust drivers for C++
- Loading DLLs in drivers
- DIY callstack symbolication

Who says hipster languages don't belong in the kernel??

STrace vs. DTrace

- **dtrace** executes at IRQL == 15, **STrace** doesn't touch IRQL – usually LOW_LEVEL
 - Memory accesses simpler
 - Enables wide range of APIs to be called in handlers
- Plugins can perform arbitrary side-effects and may call any kernel APIs
- Plugins are written in modern C++, rather than D Language
 - Faster than interpreting too!
- Symbolication of output occurs after logging, rather than within the driver.
 - For simplicity
- Only syscalls probe types are supported, FBT, PID, ETW, etc are not supported
 - Partially due to DSE being off, hypervisor features like FBT can't be implemented
 - Open to PRs for ETW!
- Zw* Apis can be monitored, dtrace filters these out.

System Architecture Choices

- Requirements
 - Must execute scripts provided from usermode somehow
 - Must be fast enough to be called every syscall
 - The script language must be pleasant and familiar to all users
 - Must execute in the kernel – zero dependencies
- Nice to haves
 - Users shouldn't need kernel development experience to use
 - Users should be able to call into the kernel for side-effects
 - Symbolic call stacks
- The Journey of **STrace's** design
 - 1) Rust w/ WASM scripts
 - 2) C++ w/ WASM scrips
 - 3) C++ w/ C++ scripts
 - 4) C++ w/ C++ scripts & Rust symbolication

WASM in the NT Kernel!

- Just bad enough of an idea it's great
- **dtrace** uses a VM for execution, it tries to be isolated. Can we do that somehow?
 - Web technologies already solved this
 - We can compile any language to WASM!
- Wasm toolchains are somewhat mature, but they all mostly focus on javascript being your build target.
 - Blockchain technologies solved this
- Enter WASMI – Parity Ethereum contracts are in WASM
 - Wasmer doesn't support **no_std** 😞
 - Not many good embedded WASM interpreters!
- Deep call stacks, exhausting the kernel stack. **KeExpandKernelStackAndCalloutEx** 😞
 - Massive local buffers??
- Too slow
 - Need a JIT, no hackable low dependency WASM JITs

Rust in the NT Kernel!

- Rust and the DDK together make an unpleasant experience
 - Microsoft only officially supports C
 - Complicated **.cargo/config** build profile needed to invoke VS linker properly
 - Complicated **build.rs** script needed
- Rust doesn't properly (easily?) generate free standing binaries
 - Unwind tables aren't reliably generated
 - Symbols like **__CxxFrameHandler3**, **__chkstk**, etc, must be stubbed – always generated ☹️
 - Handful of I.C.E.'s
- Microsoft doesn't provide Rust API projections for the kernel
 - Every single API, type, and define must be manually ported to rust ☹️
 - Rust doesn't support anonymous unions and structures; Windows uses this extensively ☹️
- Rust's **no_std** story is, complicated
 - Many libraries don't support **no_std**
 - Providing allocator mappings isn't exactly easy
 - At the time of writing, **std::ffi::CString** wasn't even supported in Rust **no_std**. This has since changed.
- **UNSAFE EVERYWHERE!**

C++ > Rust

- Rust and WASM was painful
- C++ wins
 - API and structures definitions known correct
 - Toolchain supported officially
- Can C++ host our WASM scripts instead?
 - Yea, but it's hard, no zero dependency C++ WASM interpreters – wasted weeks trying
 - Deadend.
- How else can we provide users the ability to 'script'
 - Execute blobs of code as shellcode??
 - Position Independent compiler: <https://github.com/SheLLVM/SheLLVM>
 - Little too hacky...

C++ DLLs in the NT Kernel

- Windows has a mechanism for shipping blobs of dynamic code – DLLs!
 - Not at all supported in the kernel
- Manual DLL mappers aren't that hard to write
 - Kernel isn't wildly different to usermode really
 - Remove all dependencies like the CRT, exceptions, disable stack checks and other security mechanisms
- Re-use DLL IAT and EAT for kernel APIs
- Compiler is a bit too smart with intrinsics
- It works! And it's fast! And Easy!
 - Toolchain is normal too!

```
// mycrt.h
extern "C" char* strcpy(char* a, const char* b);
#pragma intrinsic strcpy
```

```
// mycrt.cpp
#pragma function strcpy
extern "C" char* strcpy(char* a, const char* b)
{
    char* saved = a;
    while (*a++ = *b++);
    return saved;
}
```

DLLs importing kernel APIs

- DLL imports are resolved via .lib files
 - DDK injects kernel .lib files into the msvc linker path
 - Not available when not writing drivers
- Workaround - manually link a copy of ntoskrnl.lib

```
1 extern "C" __declspec(dllimport) void NTAPI ExFreePoolWithTag(  
2     PVOID P,  
3     ULONG Tag);  
4  
5 extern "C" __declspec(dllexport) void StpCallbackEntry(  
6     ULONG64 pService,  
7     ULONG32 probeId,  
8     MachineState& ctx,  
9     CallerInfo& callerinfo);
```

LogSyscallsPlugin.dll						
Module Name	Imports	OFTs	TimeDateStamp	ForwarderChain	Name RVA	FTs (IAT)
00019900	N/A	00019858	0001985C	00019860	00019864	00019868
szAnsi	(nFunctions)	Dword	Dword	Dword	Dword	Dword
ntoskrnl.exe	4	0001B280	00000000	00000000	0001B300	00010000
OFTs	FTs (IAT)	Hint	Name			
Qword	Qword	Word	szAnsi			
0000000000001B2A8	0000000000001B2A8	00C1	ExAllocatePoolWithTag			
0000000000001B2C0	0000000000001B2C0	00EE	ExFreePoolWithTag			
0000000000001B2D4	0000000000001B2D4	0C34	_snprintf			
0000000000001B2E0	0000000000001B2E0	0079	DbgkWerCaptureLiveKernelDump			

An STrace plugin DLL

```
extern "C" __declspec(dllexport) void StpCallbackEntry(ULONG64 pService, ULONG32 probeId, MachineState& ctx, CallerInfo& callerinfo)
{
    switch (probeId) {
        case PROBE_IDS::IdSetInformationFile: {
            auto hFile = (HANDLE)ctx.read_argument(0);
            auto InformationClass = ctx.read_argument(4);
            if (InformationClass == 13) { // FileDispositionInformation
                auto pInformation = (char*)ctx.read_argument(2); // 1 == DeleteFile
                if (*pInformation == 1) {
                    auto pFilePath = getFilePathFromHandle(hFile);

                    if(pFilePath) {
                        LOG_INFO("File %wZ deleted\r\n", pFilePath);
                        backupFile((wchar_t*)backup_directory, pFilePath->Name, hFile);
                        ExFreePoolWithTag(pFilePath, POOL_TAG);
                        pFilePath = nullptr;
                        LOG_INFO("File Backup Complete\r\n");
                    } else {
                        LOG_INFO("File [unknown] deleted\r\n");
                    }

                    PrintStackTrace(callerinfo);
                }
            }
            break;
        }
    }
}
```

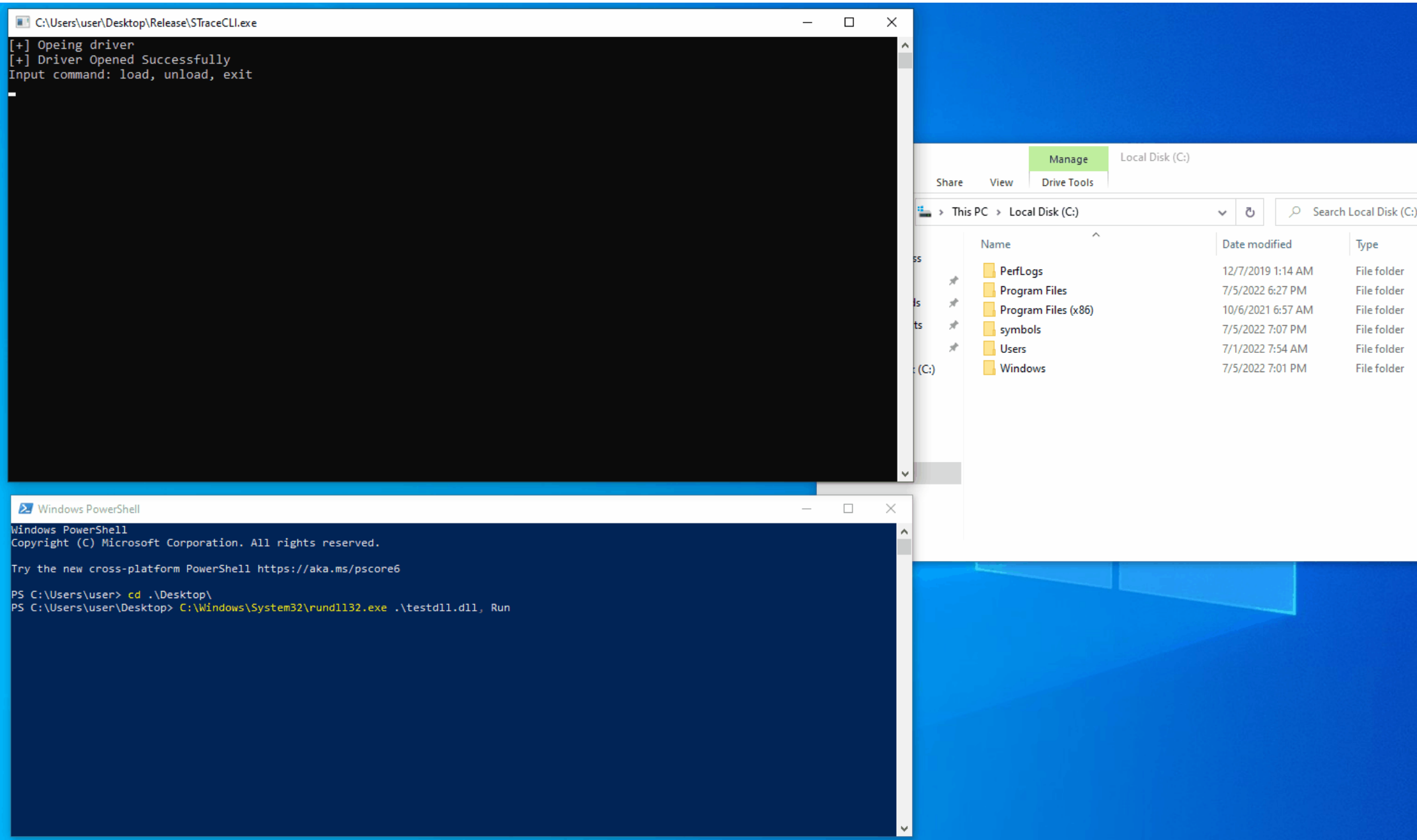
3. Handle Callbacks!

DIY Callstack Symbolication

1. Grab callstack via **RtlWalkFrameChain** + **RTL_WALK_USER_MODE_STACK**
 - ✓ Walks unwind codes just like **dtrace** does
2. Resolve callstack addresses to modules + offsets
3. Fetch and parse PDBs – Introducing **PDBReSym**!
 1. Read module PDB filename, guid, age from debug directory data
 2. Fetch PDB from http://msdl.microsoft.com/download/symbols/<pdb_filename>/<pdb_guid+pdb_age>/<pdb_filename>
 3. Cache same as WinDbg does
 4. Parse PDB
 5. Build PC line to nearest symbol RangeMap cache (binary tree).
4. Use PC line to symbol cache to resolve module name and offset pairs to symbols
5. Re-write log lines with demangled symbolic line

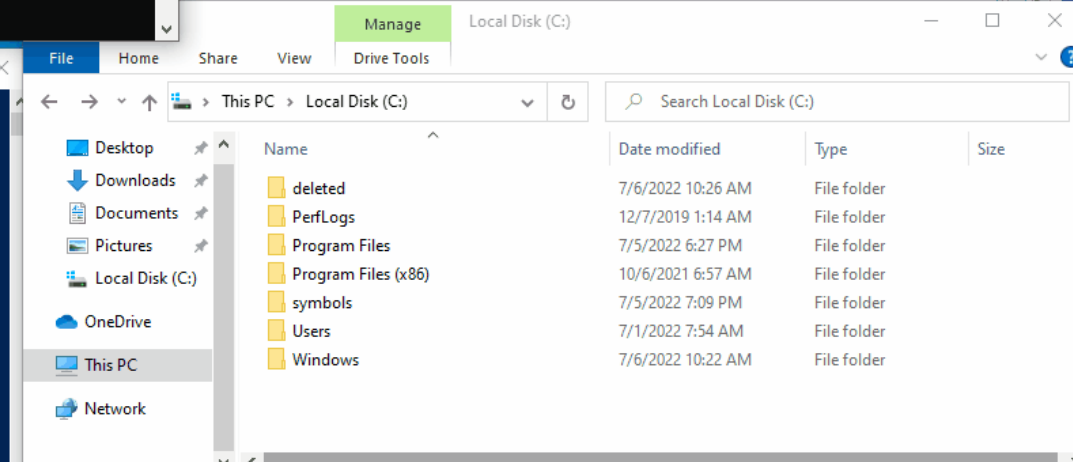
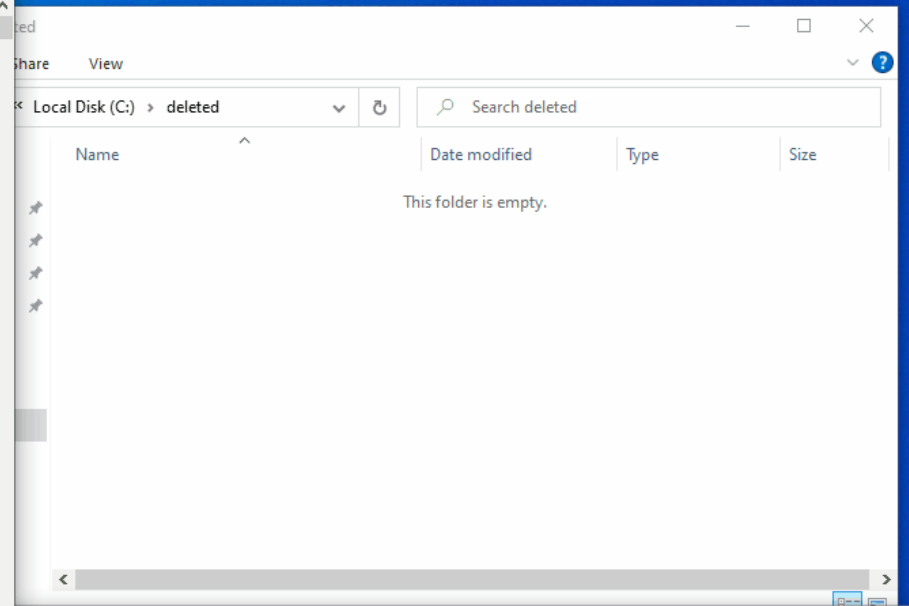
NO DIA SDK USAGE!

Demo!



```
C:\Users\user\Desktop\Release\STraceCLI.exe  
[+] Opening driver  
[+] Driver Opened Successfully  
Input command: load, unload, exit  
load  
[+] Asking for plugin  
Input command: load, unload, exit  
unload  
[+] Unloading plugin  
Input command: load, unload, exit  
load  
[+] Asking for plugin  
Input command: load, unload, exit  
unload  
[+] Unloading plugin  
Input command: load, unload, exit  
-
```

```
Windows PowerShell  
PS C:\Users\user\Desktop>
```





Thank You.

The logo features the word "MANDIANT" in a bold, sans-serif typeface. The letter "M" is rendered in red, while the remaining letters "ANDIANT" are white. The logo is centered horizontally against a dark background with a red-to-black gradient.

MANDIANT

YOUR CYBERSECURITY ADVANTAGE